

Rapport de Génie Logiciel

Projet CleMedice — Application de Gestion de Cabinet Médical

Développé par : Najim Badr Eddin & Adil Nouri & Amraji Mohsin

Framework : JavaFX 21 + Spring Boot 3.2.5

Date : Juin 2026

Introduction

CleMedice est une application desktop de gestion de cabinet médical. Elle gère les patients (CRUD avec recherche par nom, prénom ou CIN), les rendez-vous (planification, annulation, filtrage par date avec badges de statut), les consultations (liées aux rendez-vous), les ordonnances (avec liste de médicaments et génération PDF), les attestations médicales (génération PDF), la comptabilité (tableau des paiements, totaux mensuels/annuels, export Excel) et les utilisateurs (CRUD avec rôles).

Quatre rôles utilisateur sont définis : MEDECIN_PRINCIPAL (accès complet), FERMLIYAT (patients, rendez-vous, exports), ASSISTANTE (consultation seule) et AUTRE_MEDECIN (consultations, ordonnances, attestations). L'authentification utilise des jetons JWT.

Ce rapport présente le cahier des charges de CleMedice, l'organisation Scrum en 4 sprints, les trois patrons de conception implémentés (Singleton, Factory Method, Builder), la suite de 18 tests (JUnit 5, Mockito, Spring Boot Test), et les diagrammes UML du système.

Chapitre 1 — Cahier des charges de CleMedice

1.1 Contexte et objectif

CleMedice remplace la gestion papier d'un cabinet médical par une application informatisée complète. Le cabinet souhaitait centraliser la gestion des patients, rendez-vous, consultations, prescriptions et finances dans un seul outil accessible depuis un poste desktop. L'application est développée en JavaFX pour le poste client et Spring Boot pour le serveur REST.

1.2 Besoins fonctionnels

1. L'utilisateur s'authentifie via `POST /api/auth/login` avec email et mot de passe. Le backend retourne un JWT contenant l'ID, l'email et le rôle. Le token expire après 24h.
2. Le médecin peut ajouter, modifier, supprimer et rechercher des patients (nom, prénom, CIN, téléphone, date de naissance, adresse). La recherche utilise `GET /api/patients/search?keyword=`.
3. Le secrétariat peut planifier des rendez-vous avec sélection du patient, date, heure, motif. Chaque RDV affiche un badge coloré : vert pour PLANIFIE, bleu pour EFFECTUE, rouge pour ANNULE.
4. Le médecin peut annuler un RDV (statut passe à ANNULE) via `PUT /api/rendezvous/{id}/statut?statut=ANNULE`.
5. La consultation s'ouvre uniquement depuis l'écran RDV en sélectionnant un rendez-vous et en cliquant "Effectuer Consultation". Le bouton Consultation a été supprimé du tableau de bord.
6. L'ordonnance se crée uniquement après avoir sauvegardé une consultation. Le bouton "Créer Ordonnance" est désactivé tant que la consultation n'est pas enregistrée.
7. L'utilisateur ajoute des médicaments à l'ordonnance (nom, dosage, durée, instructions) et génère un PDF via `GET /api/ordonnances/{id}/pdf`. Le PDF s'ouvre dans le lecteur par défaut du système.
8. L'utilisateur génère une attestation médicale PDF via `POST /api/attestations/generate` avec sélection du patient et saisie du contenu.
9. Le MEDECIN_PRINCIPAL consulte le tableau des paiements avec totaux mensuel et annuel, ajoute des paiements et exporte les données en Excel via `GET /api/export/finance?start=&end=`.
10. Le MEDECIN_PRINCIPAL gère les utilisateurs (création, modification, suppression, réinitialisation du mot de passe) via les endpoints `GET/POST/PUT/DELETE /api/users`.

1.3 Besoins non-fonctionnels

Besoin	Valeur / Contrainte
Sécurité	JWT Bearer sur tous les endpoints sauf `/api/auth/login`. Mots de passe stockés avec BCrypt.
Performance	Temps de réponse API inférieur à 2 secondes pour 95% des requêtes.
Portabilité	JavaFX 21 garantit le fonctionnement sur Windows, macOS et Linux.
Ergonomie	Feuille de style CSS globale (`styles.css`) appliquée à toutes les vues FXML. Badges colorés pour les statuts (vert/bleu/rouge).
Base de données	H2 fichier (`./data/clemedice`) en dev . MySQL en production

1.4 Workflow imposé par l'application

11. Le patient doit exister dans le système avant toute opération.
12. Un rendez-vous est créé pour ce patient.
13. La consultation est accessible UNIQUEMENT depuis un RDV sélectionné (bouton "Effectuer Consultation").
14. L'ordonnance est créée UNIQUEMENT après la sauvegarde de la consultation (bouton "Créer Ordonnance" désactivé avant).
15. Le paiement est enregistré pour le rendez-vous correspondant.

Les boutons "Consultations" et "Ordonnances" ont été supprimés du tableau de bord pour empêcher tout accès direct et forcer le respect de ce workflow.

1.5 Architecture technique

Frontend : JavaFX 21 avec FXML pour les vues, CSS global (`styles.css`) pour le design, `java.net.http.HttpClient` pour les appels REST, Jackson pour la désérialisation JSON, token JWT stocké statiquement dans `ApiService.java`.

Backend : Spring Boot 3.2.5, Spring Security avec JWT (jjwt 0.12.5), Spring Data JPA / Hibernate, iText Core 8.0.4 pour la génération PDF, Apache POI 5.2.5 pour l'export Excel.

Contrôleurs frontend : `LoginController`, `DashboardController`, `PatientsController`, `RendezVousController`, `ConsultationController`, `OrdonnanceController`, `AttestationController`, `FinanceController`, `UsersController`.

Endpoints backend principaux :

Méthode	Endpoint	Rôle requis
POST	`/api/auth/login`	Aucun
GET	`/api/patients`	Tous
GET	`/api/patients/search?keyword=`	Tous
POST	`/api/patients`	MEDECIN_PRINCIPAL, FERMLIYAT
PUT	`/api/patients/{id}`	MEDECIN_PRINCIPAL, FERMLIYAT
DELETE	`/api/patients/{id}`	MEDECIN_PRINCIPAL
GET	`/api/rendezvous`	Tous
GET	`/api/rendezvous/date?date=`	Tous
POST	`/api/rendezvous`	Tous
PUT	`/api/rendezvous/{id}/statut?statut=`	Tous
GET	`/api/consultations/rendezvous/{rdvId}`	Tous
POST	`/api/consultations`	Tous
PUT	`/api/consultations/{id}`	Tous
POST	`/api/ordonnances`	Tous

GET	`/api/ordonnances/{id}/pdf`	Tous
POST	`/api/attestations/generate`	Tous
GET	`/api/finance`	MEDECIN_PRINCIPAL
POST	`/api/finance`	MEDECIN_PRINCIPAL
GET	`/api/finance/summary?annee=&mois=`	MEDECIN_PRINCIPAL
GET	`/api/export/finance?start=&end=`	MEDECIN_PRINCIPAL
GET/POST/PUT/DELETE	`/api/users/{id}`	MEDECIN_PRINCIPAL

1.6 Modèle de données

7 tables composent la base de données :

Table	Champs
users	id, nom, email, password (BCrypt), role (MEDECIN_PRINCIPAL / FERMLIYAT / ASSISTANTE / AUTRE_MEDECIN), enabled
patients	id, nom, prenom, cin (unique), telephone, date_naissance, adresse, created_at
rendez_vous	id, patient_id (FK → patients), medecin_id (FK → users), date, heure, topic, statut (PLANIFIE / EFFECTUE / ANNULE), created_at
consultation_details	id, rendez_vous_id (FK → rendez_vous, OneToOne), description, observations, cas_patient, date
ordonnances	id, consultation_id (FK → consultation_details), cas_patient, date
medicaments	id, ordonnance_id (FK → ordonnances), nom, dosage, duree, instructions
paiements	id, rendez_vous_id (FK → rendez_vous, OneToOne), montant (BigDecimal), date, mode_paiement (ESPECES / CHEQUE / VIREMENT / CARTE_BANCAIRE), statut, notes

Chapitre 2 — Modèle de développement utilisé

2.1 Organisation en sprints

Le développement a suivi Scrum avec 4 sprints de 2 semaines chacun.

Sprint	Semaines	Fonctionnalités développées	Livrable
1	1-2	Authentification JWT, CRUD patients, structure Spring Boot + JavaFX	Login + écran Patients fonctionnel
2	3-4	Rendez-vous avec filtrage par date, badges de statut (PLANIFIE/EFFECTUE/ANNULE)	Écran RDV complet avec navigation
3	5-6	Consultations liées aux RDV, ordonnances avec médicaments, génération PDF	Workflow Patient→RDV→Consultation→Ordonnance complet
4	7-8	Gestion financière, attestations PDF, utilisateurs, 18 tests.	Application complète + tests

2.2 Répartition des tâches (RACI simplifié)

Composant	Najim	Adil	Mohsin
Authentification / Security / JWT	R/A	C	I
Structure JavaFX, navigation, CSS global	R/A	I	C
ApiService (client HTTP)	R/A	C	I
Patients (CRUD + recherche)	R/A	C	I
Rendez-vous (CRUD + filtrage)	R/A	C	I
Entités JPA / Repositories	C	R/A	I
Consultations	A	R	I
Ordonnances + PDF	C	R/A	I
Attestations PDF	I	R/A	C
Finance (paiements, totaux, export Excel)	C	R/A	I
Gestion utilisateurs	R/A	C	I
DataInitializer (seed)	R/A	C	I
Tests (JUnit, Mockito, Intégration)	R	A	C/I

Chapitre 3 — Design Patterns implémentés

3.1 Singleton — AppConfig

Fichier : `frontend/src/main/java/com/cabinet/ui/config/AppConfig.java`

```
public class AppConfig {
    private static volatile AppConfig instance;

    private String apiBaseUrl = "http://localhost:8080/api";
    private String appName = "CleMedice";
    private String version = "1.0.0";
    private int timeoutSeconds = 10;

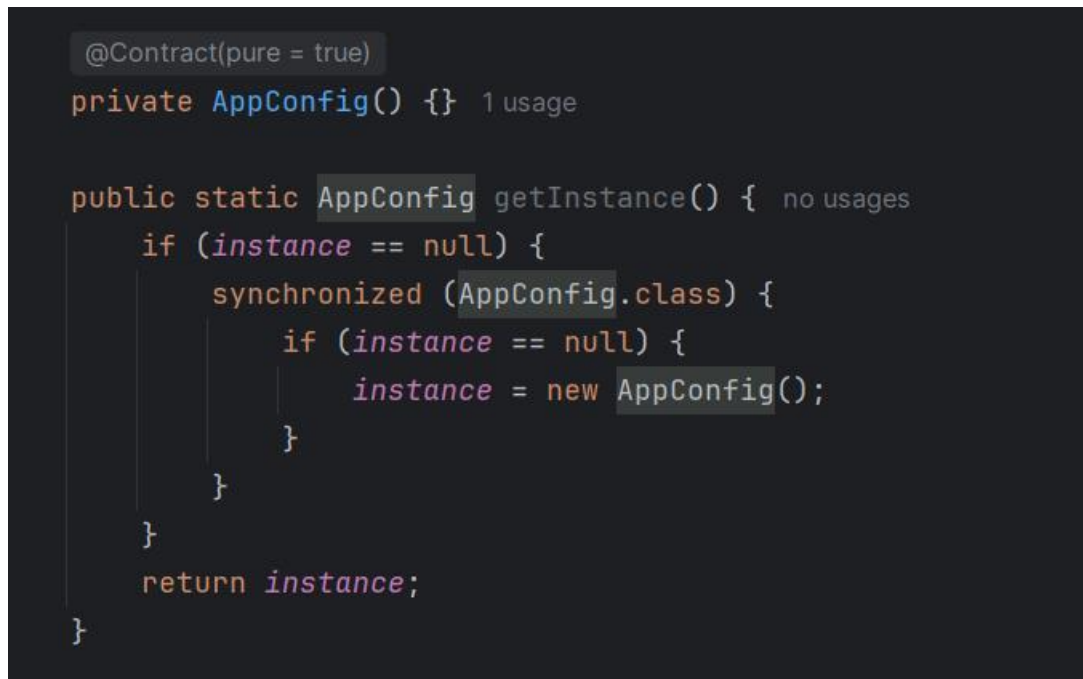
    private AppConfig() {}

    public static AppConfig getInstance() {
        if (instance == null) {
```

```

        synchronized (AppConfig.class) {
            if (instance == null) {
                instance = new AppConfig();
            }
        }
    }
    return instance;
}
// getters et setters
}

```



```

@Contract(pure = true)
private AppConfig() {} 1 usage

public static AppConfig getInstance() { no usages
    if (instance == null) {
        synchronized (AppConfig.class) {
            if (instance == null) {
                instance = new AppConfig();
            }
        }
    }
    return instance;
}

```

Dans CleMedice, tous les contrôleurs JavaFX accèdent à la même URL de base de l'API (`http://localhost:8080/api`) via `AppConfig.getInstance().getApiBaseUrl()`. Le double-checked locking avec `volatile` garantit qu'un seul thread crée l'instance, même si plusieurs contrôleurs chargent simultanément.

3.2 Factory Method — DocumentFactory

Fichier : `backend/src/main/java/com/cabinet/factory/DocumentFactory.java`

```

public enum DocumentType {
    ORDONNANCE, ATTESTATION,
    CERTIFICAT_ARRET_TRAVAIL, CERTIFICAT_APTITUDE
}

public static Map<String, String> createDocument(
    DocumentType type, String patientNom,
    String patientPrenom, String contenu) {
    // switch sur type, retourne une Map avec:
    // - type, titre, entete differents selon le type
    // - patientNom, patientPrenom, date, contenu communs
}

```

```

public class DocumentFactory { 18 usages
    public enum DocumentType { 11 usages
        ORDONNANCE, 4 usages
        ATTESTATION, 2 usages
        CERTIFICAT_ARRET_TRAVAIL, 2 usages
        CERTIFICAT_APTITUDE 2 usages
    }

    public static @NotNull Map<String, String> createDocument( 8 usages
        @NotNull DocumentType type,
        String patientNom,
        String patientPrenom,
        String contenu) {

        Map<String, String> doc = new HashMap<>();
        doc.put("patientNom", patientNom);
        doc.put("patientPrenom", patientPrenom);
        doc.put("date", LocalDate.now().toString());
        doc.put("contenu", contenu);

        switch (type) {
            case ORDONNANCE -> {
                doc.put("type", "ORDONNANCE");
                doc.put("titre", "Ordonnance Médicale");
                doc.put("entete", "Je soussigné, prescris au patient " +
                    patientNom + " " + patientPrenom + " :");
            }
            case ATTESTATION -> {

```

Quatre types de documents partagent la même structure (`patientNom`, `date`, `contenu`) mais diffèrent par `type`, `titre` et `entete`. Ordonnance produit "Ordonnance Médicale", Attestation produit "Attestation Médicale", etc. La factory centralise cette logique : ajouter un nouveau type revient à ajouter une valeur à l'enum et un `case` dans le `switch`.

3.3 Builder — RendezVousRequest

Fichier : `frontend/src/main/java/com/cabinet/ui/model/RendezVousRequest.java`

```

public class RendezVousRequest {
    private final Long patientId;
    private final Long medecinId;
    private final LocalDate date;
    private final LocalTime heure;
    private final String topic;
    private final String statut;

    private RendezVousRequest(Builder b) { /* copie depuis builder */ }

    public static class Builder {
        private Long patientId, medecinId;
        private LocalDate date;

```

```

private LocalTime heure;
private String topic = "";
private String statut = "PLANIFIE";

public Builder patientId(Long v) { this.patientId = v; return this; }
public Builder medecinId(Long v) { this.medecinId = v; return this; }
public Builder date(LocalDate v) { this.date = v; return this; }
public Builder heure(LocalTime v) { this.heure = v; return this; }
public Builder topic(String v) { this.topic = v; return this; }
public RendezVousRequest build() {
    if (patientId == null) throw new IllegalStateException("patientId requis");
    if (medecinId == null) throw new IllegalStateException("medecinId requis");
    if (date == null)      throw new IllegalStateException("date requise");
    if (heure == null)     throw new IllegalStateException("heure requise");
    return new RendezVousRequest(this);
}
}
}

```

```

public class RendezVousRequest { 2 usages
    public static class Builder { 7 usages
        private Long      patientId; 3 usages
        private Long      medecinId; 3 usages
        private LocalDate  date; 3 usages
        private LocalTime  heure; 3 usages
        private String     topic = ""; 2 usages
        private String     statut = "PLANIFIE"; 2 usages

        public Builder patientId(Long patientId) { no usages
            this.patientId = patientId;
            return this;
        }
        public Builder medecinId(Long medecinId) { no usages
            this.medecinId = medecinId;
            return this;
        }
        public Builder date(LocalDate date) { no usages
            this.date = date;
            return this;
        }
        public Builder heure(LocalTime heure) { no usages
            this.heure = heure;
            return this;
        }
        public Builder topic(String topic) { no usages
            this.topic = topic;
            return this;
        }
        public Builder statut(String statut) { no usages
            this.statut = statut;
            return this;
        }
        public RendezVousRequest build() { no usages

```

Utilisation typique dans le contrôleur RDV :

```
RendezVousRequest req = new RendezVousRequest.Builder()
    .patientId(selectedPatientId)
    .medecinId(getLoggedInMedecinId())
    .date(datePicker.getValue())
    .heure(LocalTime.parse(timeField.getText()))
    .topic(topicField.getText())
    .build();
```

La création d'un RDV nécessite 6 champs dont 4 obligatoires. Le builder valide les champs manquants dans `build()` avant de créer l'objet. `statut` a une valeur par défaut "PLANIFIE".

3.4 Tableau récapitulatif

Pattern	Classe	Package	Problème résolu dans CleMedice
Singleton	`AppConfig`	`com.cabinet.ui.config`	Configuration centralisée (URL API, timeout) accessible par tous les contrôleurs JavaFX
Factory Method	`DocumentFactory`	`com.cabinet.factory`	Création de 4 types de documents médicaux avec structure commune mais titres et en-têtes différents
Builder	`RendezVousRequest`	`com.cabinet.ui.model`	Construction d'une requête RDV avec 6 champs, validation des 4 champs obligatoires avant création

Chapitre 4 — Tests

4.1 PatientServiceTest — Tests unitaires (Mockito)

Fichier : `backend/src/test/java/com/cabinet/service/PatientServiceTest.java`

mvn test -Dtest=PatientServiceTest :

```
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.371 s -- in com.cabinet.service.PatientServiceTest
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 9.049 s
[INFO] Finished at: 2026-06-07T20:03:22+01:00
[INFO]
```

4.2 DocumentFactoryTest – Tests unitaires

****Fichier : **** `backend/src/test/java/com/cabinet/factory/DocumentFactoryTest.java`

mvn test -Dtest=DocumentFactoryTest :

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.cabinet.factory.DocumentFactoryTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.203 s -- in com.cabinet.factory.DocumentFactoryTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.189 s
[INFO] Finished at: 2026-06-07T20:04:44+01:00
[INFO] -----
```

4.3 PatientControllerIntegrationTest – Tests d'intégration

***Fichier :** `backend/src/test/java/com/cabinet/controller/PatientControllerIntegrationTest.java`*

mvn test -Dtest=PatientControllerIntegrationTest :

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 18.09 s -- in com.cabinet.controller.PatientControllerIntegrationTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 25.272 s
[INFO] Finished at: 2026-06-07T20:07:02+01:00
[INFO] -----
```

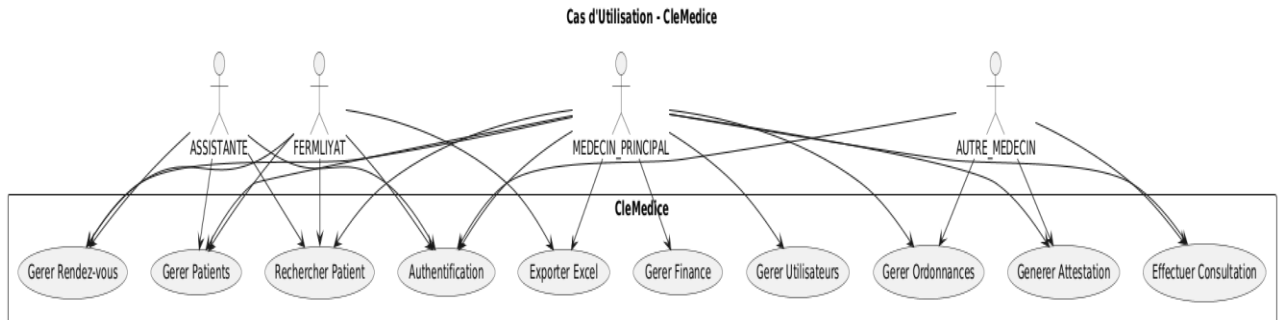
4.4 Résultats globaux

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 21.27 s -- in com.cabinet.controller.PatientControllerIntegrationTest
[INFO] Running com.cabinet.factory.DocumentFactoryTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in com.cabinet.factory.DocumentFactoryTest
[INFO] Running com.cabinet.factory.SingletonPatternTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.016 s -- in com.cabinet.factory.SingletonPatternTest
[INFO] Running com.cabinet.service.PatientServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.789 s -- in com.cabinet.service.PatientServiceTest
[INFO] Results:
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 28.639 s
[INFO] Finished at: 2026-06-07T12:10:26+01:00
[INFO] -----
```

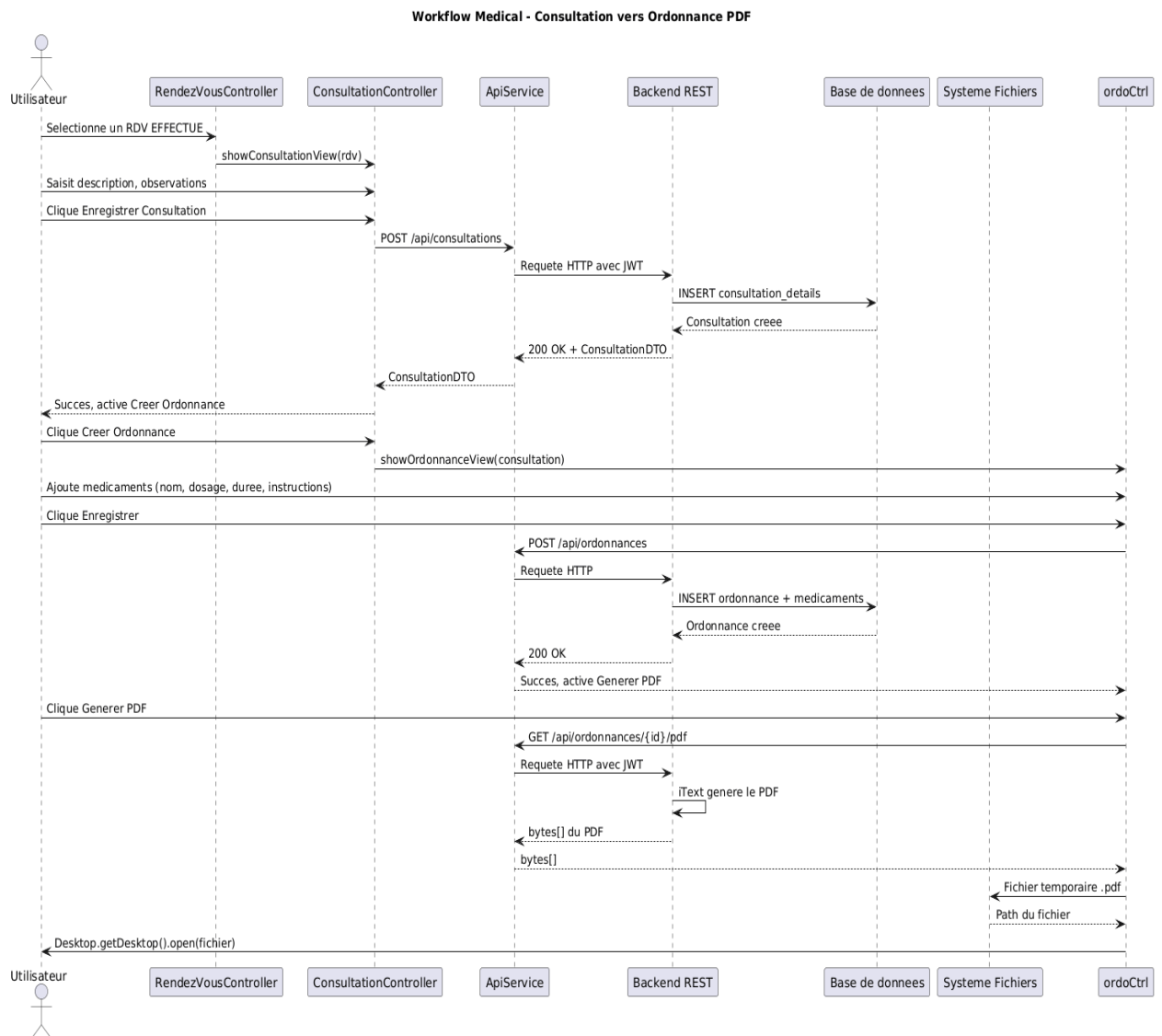
*Commande d'exécution : ** `mvn test` (dans Le dossier `backend/`)*

Chapitre 4 — Diagrammes UML

5.1 Diagramme de cas d'utilisation – CleMedice

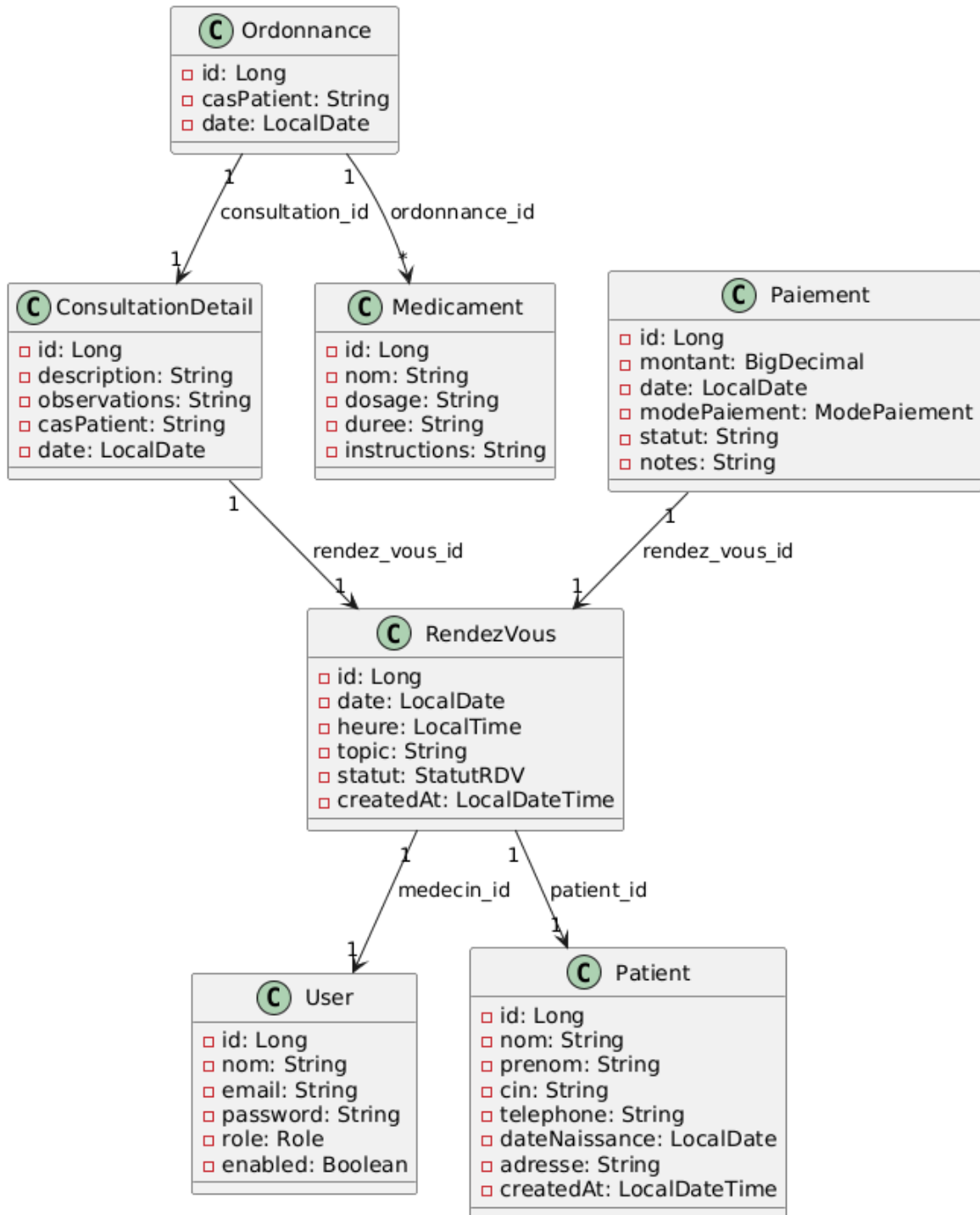


5.2 Diagramme de séquence – Workflow médical principal

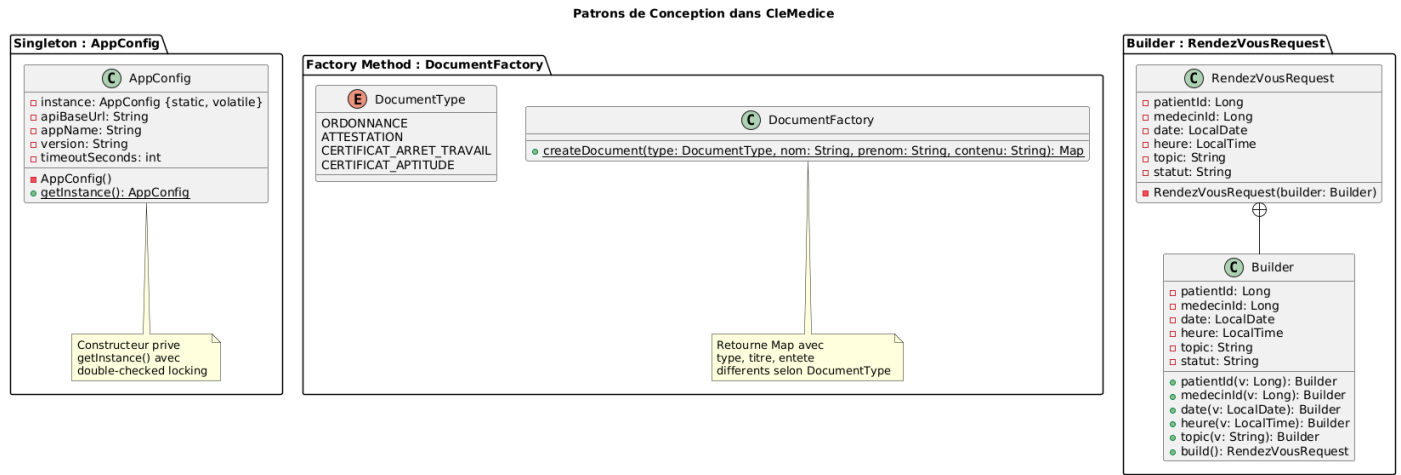


5.3 Diagramme de classes – Entités principales

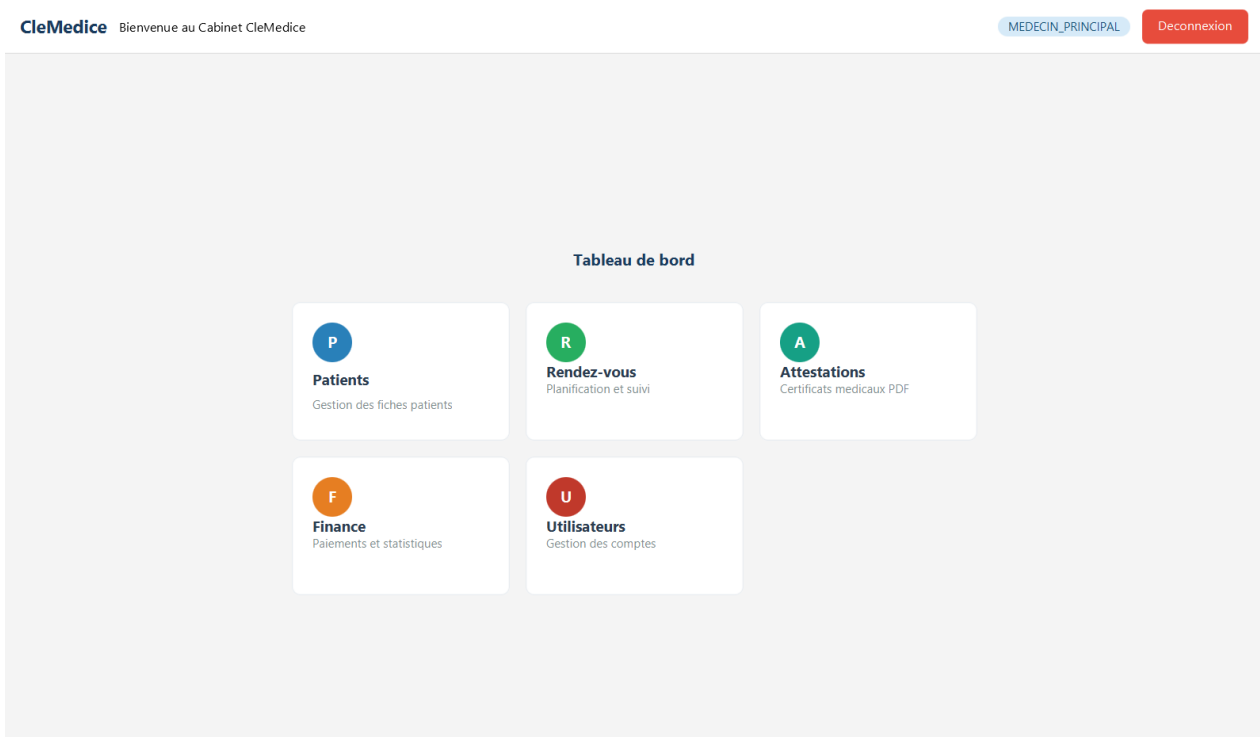
Diagramme de Classes - Entites CleMedice



5.4 Diagramme de classes – Design Patterns



Chapitre 6 — application



[< Retour](#)

Gestion des Utilisateurs

[illegible]

Ajouter

Modifier

[Reset Mot de passe](#)

Supprimer

[< Retour](#)

Gestion des Patients

Rechercher

[illegible]

Ajouter

Modifier

Supprimer

[< Retour](#)

Rendez-vous

6/6/2026

Aujourd'hui

[illegible]

Nouveau RDV

Effectuer Consultation

Ajouter Paiement

Annuler RDV

[< Retour](#)

Consultation

Mohsin Amraji Amraji

Date RDV: 2026-06-06

Description

Occupation: Taxi Driver
Marital Status: Married
Weight: 82 kg
Height: 175 cm

Observations

A screenshot of a patient information window. The window has a title bar with a green icon and the word 'Success'. Below the title bar, the text reads: 'Patient is conscious, alert, and oriented.', 'Temperature: 37.2°C', 'Blood Pressure: 145/90 mmHg', and 'Heart Rate: 88 bpm'. At the bottom of the window, there is a 'Message' button with a blue circular icon containing a white 'i'.

Cas Patient

Patient reports fatigue, headache, and mild dizziness	Concomitant symptoms	
---	----------------------	--

Date _____

6/6/2026

Enregistrer Consultation

Créer Ordonnance

Succes

Message 

Consultation enregistree

OK

Ordonnance

Cas Patient

Medicaments

Finance

Total ce mois

Total cette annee

Annee

2026

Mois

Juin

Actualiser

Paielements

[illegible]